PG Manager: A Secure Full-Stack Web Application for Paying Guest House Administration

[Student Name 1]	[Email ID]	[Mobile Number]
[Student Name 2]	[Email ID]	[Mobile Number]
[Student Name 3]	[Email ID]	[Mobile Number]
[Student Name 4]	[Email ID]	[Mobile Number]

GitHub Link: [repository URL] Project Deployment Link: [deployment URL] Project Guide Name: [Guide Name]

Abstract

Managing a Paying Guest (PG) house involves coordinating rooms, tenant records, rent collection with proof verification, complaint resolution, expense tracking, and communication through notices. Traditional manual or spreadsheet-based approaches introduce errors, lack access control, and provide no audit trail. This project presents PG Manager, a secure full-stack web application that digitizes the complete administrative workflow for small residential rental properties. The system employs a React 18 single-page frontend communicating with a Node.js/Express backend over HTTPS, backed by PostgreSQL for relational data storage. Security is addressed through layered defences: JWT-based authentication in httpOnly SameSite=Strict cookies, bcrypt password hashing with cost factor 12, per-account brute-force lockout, CSRF protection via custom headers, Content Security Policy headers, role-based access control separating administrator and tenant permissions, parameterized SQL queries, Zod schema validation on all inputs, and magic-byte file type verification for uploaded payment screenshots. The application supports real-time dashboards computing occupancy rates, rent collection progress, overdue tenant identification, and six-month income-versus-expense trend visualization. Fifteen automated API integration tests validate authentication, authorization, business logic, file upload security, and data isolation, all passing successfully. The system is deployable on cloud infrastructure using Docker containers or AWS EC2 with RDS PostgreSQL, demonstrating practical viability for small-scale property administrators seeking a self-hosted management solution.

Keywords—Full-stack web application, Role-based access control, JWT authentication, PostgreSQL, Cloud-native deployment, Secure file upload

I. INTRODUCTION

The rapid urbanization in India has created a substantial demand for organized residential rental accommodations, particularly Paying Guest (PG) houses that cater to students and working professionals. Managing such properties involves a complex interplay of room allocation, tenant registration, periodic rent collection with payment proof verification, complaint handling, expense recording, and tenant communication. Many small-scale PG operators still rely on paper registers, spreadsheets, or messaging applications for these tasks, leading to data inconsistency, lack of accountability, limited visibility into financial health, and no structured audit trail [4].

Recent literature demonstrates growing interest in web-based property management. Francis and Frimpong [4] designed a house rental management system using web technologies to streamline tenant onboarding and payment tracking. Saputra et al. [6] implemented a boarding room management system following the Waterfall methodology, confirming that digital systems reduce administrative overhead. In the security domain, Bucko et al. [1] proposed enhancing JWT authentication with user behavior analysis, while Rana et al. [5] validated the efficacy of JWT combined with HMAC SHA-256 for web application security. Wichmann et al. [2] addressed file upload vulnerabilities through server-side detection middleware.

Despite these advances, existing solutions exhibit several gaps. Many property management systems prioritize functionality over security, lacking comprehensive protections against common web vulnerabilities such as cross-site request forgery, cross-site scripting, and injection attacks [7]. Cloud-native deployment considerations are often overlooked in academic prototypes [3]. Furthermore, systems that accept file uploads frequently trust client-declared

MIME types rather than validating actual file content [2], and role-based access control is often implemented without live account status verification, allowing deactivated users to retain access until token expiry [1].

PG Manager addresses these gaps by implementing a security-hardened, full-stack management system specifically designed for the PG house operational context. The system provides separate authenticated interfaces for property owners (administrators) and tenants, enforces role-based access at every API endpoint, validates uploaded payment screenshots through magic-byte inspection rather than MIME headers, re-checks account status against the database on every request, and maintains a complete audit log of all state-changing operations. The application is deployable through Docker containers or cloud infrastructure with HTTPS termination, making it practical for real-world use.

The key contributions of this project are:

- Design and implementation of a complete full-stack PG management system covering rooms, tenants, rent payments with proof verification, complaints, notices, expenses, and audit logging.
- A layered security architecture implementing JWT authentication with httpOnly cookies, CSRF protection, rate limiting, Content Security Policy, parameterized queries, input schema validation, and magic-byte file verification.
- Role-based access control with live database verification ensuring immediate effect of account deactivation and preventing privilege escalation.
- Data-driven dashboards computing occupancy, collection progress, overdue identification, and six-month financial trends for administrative decision-making.
- A comprehensive automated test suite of fifteen integration tests validating security properties, business logic correctness, and data isolation.
- Cloud-deployable architecture supporting Docker Compose, AWS EC2 with RDS, and process management through PM2.

The remainder of this paper is organized as follows. Section II presents the related work and literature survey. Section III describes the methodology including architecture, data handling, and algorithms. Section IV details the implementation across software requirements, frontend, and backend. Section V reports results and analysis from testing. Section VI concludes the paper.

II. RELATED WORK / LITERATURE SURVEY

Table 1. Related Work for a given problem statement

Paper No.	Methodology / Algorithm / Technology	Dataset	Results	Limitations
[1]	JWT-based authentication enhanced with user behavior history analysis; trust scoring mechanism for session validation	Simulated user sessions	Improved authentication trust accuracy; reduced unauthorized access by incorporating behavioral anomalies	Requires accumulation period for behavioral profiles; not tested against sophisticated session hijacking
[2]	FileUploadChecker: server-side middleware detecting malicious uploads at request level using content analysis	Curated malicious file corpus	Detected and sanitized 94% of test malicious uploads; zero false negatives on executable payloads	Adds processing latency per upload; framework-specific implementation; limited to known attack patterns
[3]	Comprehensive survey of cloud-native computing lifecycle: containerization, orchestration, microservices, serverless	Literature review (170+ papers)	Taxonomy of cloud-native patterns; identified deployment, scalability, and observability challenges	Survey scope limits implementation depth; rapidly evolving field may reduce findings validity
[4]	Web-based house rental management system using PHP/MySQL; CRUD operations for tenants, payments, and complaints	Primary data from local property managers	Functional prototype demonstrating digitized rental workflow; reduced manual record-keeping time	No security analysis reported; single-role access; no file upload handling; no cloud deployment
[5]	Descriptive study evaluating JWT with HMAC SHA-256 for web application security; token lifecycle analysis	Not applicable (analytical study)	JWT/HMAC-SHA-256 provides strong integrity and confidentiality against common threats when properly configured	Token revocation remains architecturally complex; study does not propose novel mechanism
[6]	Waterfall SDLC methodology for web-based boarding room management; PHP,	Primary requirements from boarding house operators	Successfully delivered functional system for room booking, tenant records, and fee	Waterfall rigidity limits adaptation to changing requirements; no security

	MySQL, Bootstrap technology stack		management	hardening discussed
[7]	Microservices-driven automation in full-stack development; containerized deployment with CI/CD pipelines	Industrial case studies	Demonstrated improved scalability and deployment velocity through service decomposition	Microservices complexity overhead unjustified for small single-server applications; operational cost increase

Synthesizing the above literature, several common themes emerge. Web-based management systems effectively digitize property administration workflows [4][6], but academic prototypes frequently neglect security hardening. JWT-based authentication provides stateless, scalable session management [1][5], yet requires careful implementation to address revocation and behavioral validation. File upload security demands content-level verification beyond MIME headers [2]. Cloud-native deployment patterns [3][7] offer operational benefits but introduce complexity that must be justified by scale. The engineering gap addressed by PG Manager is the integration of comprehensive security measures within a practical, deployable management system tailored to the specific operational needs of small residential rental properties.

III. METHODOLOGY

A. Architecture / Flow Diagram

The PG Manager system follows a three-tier web application architecture comprising a client layer, application server layer, and data persistence layer, with a reverse proxy handling transport security. The architecture, depicted in Fig. 1, reflects the actual implementation boundaries in the codebase.

The client layer is a React 18 single-page application built with Vite, serving distinct views for administrators (Dashboard, Rooms, Tenants, Payments, Complaints, Notices, Expenses, Audit) and tenants (Home, Rent Payment, Complaints, Notices, Profile). All API communication uses the browser fetch API with credentials included, automatically sending the httpOnly JWT cookie and the required X-Requested-With header for CSRF protection.

The application server layer runs Node.js with Express, processing requests through a security middleware chain before reaching route handlers. The middleware stack includes Helmet for Content Security Policy and HSTS headers, express-rate-limit for throttling (600 API requests and 30 auth requests per 15-minute window), a custom CSRF guard requiring the X-Requested-With header on all state-changing methods, cookie-parser for JWT extraction, and express.json with a 20KB body limit. Route handlers delegate to business logic modules that perform Zod schema validation, database operations through parameterized SQL, and audit logging.

The data layer uses PostgreSQL accessed through a connection pool (pg library, maximum 10 connections). Payment proof images are stored as binary large objects (BYTEA) directly in the database rather than on the filesystem, ensuring they survive container redeployments. The schema defines seven tables: users, rooms, payments, complaints, announcements, expenses, and audit_log.

Deployment places Nginx as a reverse proxy terminating HTTPS connections with Let's Encrypt certificates, forwarding to the Express server on port 5000. The entire stack is containerizable through Docker Compose or deployable on AWS EC2 with a private RDS PostgreSQL instance.

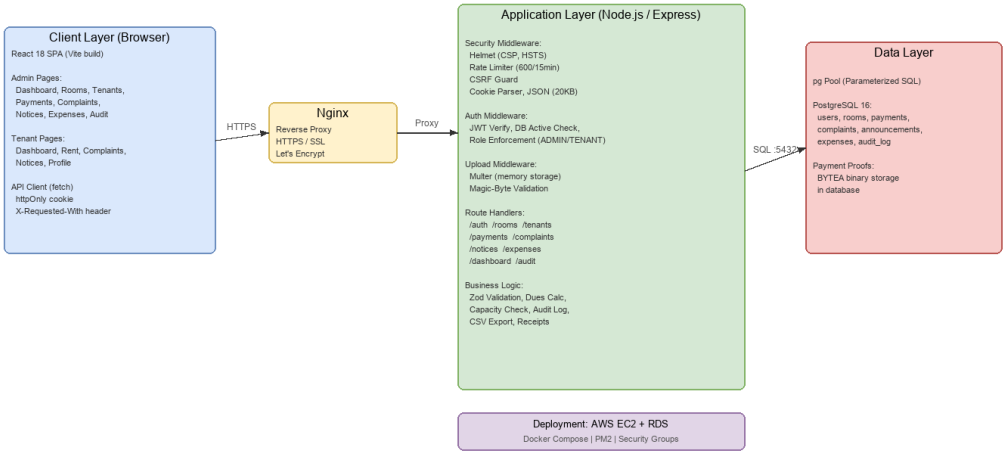


Fig. 1. Architecture of the PG Manager system

Fig. 1. Architecture of the PG Manager system

B. Dataset / Input Used

PG Manager is an operational management system rather than a data-analysis or machine learning pipeline, so it does not consume a conventional static dataset. The system processes live user input structured according to the application's domain model. Input sources include:

- Room records: number, floor, type (AC/NON_AC), capacity (1-20), monthly rent, security deposit.
- Tenant registrations: name, email, password, phone number, optional emergency contact.
- Rent payment submissions: month identifier (YYYY-MM format), amount, and a UPI transaction screenshot as a JPG, PNG, or WebP image up to 2 MB.
- Complaints: title, description, category (Maintenance, Cleaning, Electrical, Plumbing, Food, Noise, Other), priority (Low, Medium, High).
- Notices: title, body text, pin status.
- Expense entries: title, category (Electricity, Water, Food, Maintenance, Salary, Internet, Other), amount, date spent.

The system's demonstration mode uses an in-memory PostgreSQL emulator (pg-mem) populated with sample data including 6 rooms across 3 floors, 8 tenants, approximately 20 payment records spanning 4 months, 3 complaints, 2 notices, and 8 expense entries, providing realistic evaluation conditions without requiring a persistent database installation.

C. Input Preprocessing Steps

The system applies multiple validation and sanitization layers to all incoming data, functioning as the input preprocessing pipeline:

1. Schema Validation with Zod: Every request body is parsed through a Zod schema before reaching business logic. Schemas enforce type constraints (string length bounds, integer ranges, enum values), format requirements (email regex, date format YYYY-MM-DD, month format YYYY-MM, phone pattern 7-15 digits with optional +, -, space), and presence of required fields. Invalid requests receive HTTP 400 with the first validation error message.

2. Authentication and Session Verification: The JWT token extracted from the httpOnly cookie is cryptographically verified against the server secret. The decoded user ID is then looked up in the database to confirm the account exists and is active, preventing access by deactivated users holding unexpired tokens.

3. CSRF Header Check: All state-changing HTTP methods (POST, PUT, PATCH, DELETE) must carry the custom X-Requested-With: pg-manager header. Requests without this header are rejected with HTTP 403, providing defence-in-depth alongside SameSite=Strict cookies.

4. File Upload Validation: Payment proof images undergo two-stage verification. First, the MIME type declared by the browser is checked against the allowed set (image/jpeg, image/png, image/webp). Second, the actual file content is inspected by reading magic bytes from the buffer: JPEG (FF D8 FF), PNG (89 50 4E 47 0D 0A 1A 0A), or WebP (RIFF...WEBP). Files passing the MIME check but failing signature verification are rejected, preventing content-type spoofing attacks.

5. Rate Limiting: The API enforces 600 requests per 15 minutes for general endpoints and 30 for authentication endpoints, preventing brute-force and denial-of-service attempts.

6. SQL Parameterization: All database queries use parameterized placeholders ($1, $2, ...) rather than string concatenation, preventing SQL injection. Column names in dynamic UPDATE statements come from a server-side whitelist, never from user input.

7. Output Sanitization: CSV export cells starting with =, +, -, or @ are prefixed with a single quote to neutralize spreadsheet formula injection. The React frontend escapes all rendered content, preventing XSS. File responses include X-Content-Type-Options: nosniff and a restrictive CSP.

D. Algorithms / Techniques Used

1. JWT-Based Stateless Authentication: JSON Web Tokens encode the user ID and role, signed with HMAC SHA-256 using a server secret. Tokens expire after 8 hours. The approach avoids server-side session storage while maintaining verifiable identity claims [1][5]. The token is transmitted via httpOnly, SameSite=Strict, Secure cookies, preventing JavaScript access and cross-site leakage.

2. bcrypt Password Hashing: Passwords are hashed with the bcrypt algorithm at cost factor 12 (2^12 = 4096 rounds of the EksBlowfish key schedule). This provides adaptive one-way hashing that resists GPU-accelerated brute-force attacks. A dummy hash comparison ensures constant response time for unknown email addresses, preventing user enumeration.

3. Per-Account Brute-Force Lockout: After 5 consecutive failed login attempts, the account is locked for 15 minutes by setting a locked_until timestamp. Successful login resets the counter. This complements the IP-based rate limiter to provide targeted protection.

4. Role-Based Access Control (RBAC): Two roles (ADMIN and TENANT) are enforced through middleware that checks the authenticated user's role against the endpoint's requirement. Tenants can only access their own payment records, proof images, and complaints through ownership verification in every query. The admin account is provisioned from environment variables at startup, while public registration creates only TENANT accounts.

5. Dues Computation Algorithm: The system calculates a tenant's unpaid rent months by determining the range from their join month to the current billing month (considering the 5th-of-month due day), excluding months with APPROVED or PENDING payments, capped at the most recent 12 months. The total due equals unpaid month count multiplied by the room's monthly rent.

6. Magic-Byte File Signature Detection: Binary file headers are matched against known image format signatures to verify file type independently of client-declared MIME types [2]. This prevents attacks where malicious content is disguised with an image extension.

7. Connection Pooling: The pg library maintains a pool of up to 10 database connections, reusing established connections across requests to amortize TCP and authentication overhead.

IV. IMPLEMENTATION

A. Software / Cloud Platform / Hardware Requirements

Component	**Technology**
Programming Language	JavaScript (Node.js 20+, ES2022)
Frontend Framework	React 18, Vite 5 build tool
Backend Framework	Express 4.x
Database	PostgreSQL 16 (via pg client library)
Authentication	jsonwebtoken 9.x, bcryptjs 2.x
Validation	Zod 3.x schema library
Security Middleware	Helmet 7.x, express-rate-limit 7.x

File Handling	Multer 1.x (memory storage)
Testing	Node.js built-in test runner, pg-mem (in-memory Postgres)
Deployment	Docker 24+, AWS EC2 (t3.micro), RDS PostgreSQL, Nginx, PM2, Let's Encrypt
Hardware (Development)	Any system with Node.js 20+; 4 GB RAM sufficient for development

The verified minimum for development is Node.js 20 and a PostgreSQL instance (or pg-mem for testing/demo). Production deployment on AWS uses a t3.micro EC2 instance (1 vCPU, 1 GB RAM) within the free tier, paired with a db.t3.micro RDS instance. No GPU or specialized hardware is required as the system performs no machine learning inference.

B. Front End Development and Results

The frontend is a React 18 single-page application with no external UI library, using hand-written CSS for styling. The application structure follows a component-per-page pattern organized in the frontend/src/pages/ directory.

The main App component manages global state (authenticated user, current page via hash-based routing, dark/light theme toggle) and renders either the authentication screen (login/register) or the role-appropriate navigation shell. Administrators see eight navigation items (Dashboard, Rooms, Tenants, Payments, Complaints, Notices, Expenses, Activity); tenants see four (Home, Rent, Complaints, Notices) plus Profile.

The API client module (api.js) wraps fetch calls with automatic credential inclusion, the X-Requested-With CSRF header, JSON serialization, and centralized 401 handling that triggers re-authentication. Each page component uses a useLoad custom hook for data fetching with loading, error, and empty states.

The admin dashboard displays computed statistics (occupancy percentage, rent collected versus expected, pending approvals, open complaints), a six-month income-versus-expense bar chart, and an overdue tenant list. The tenant dashboard shows their room details, current month payment status, total dues, and a call-to-action for unpaid rent. The payments page supports file upload via FormData, status filtering, and admin approval/rejection workflow. All pages are responsive and support dark mode via CSS custom properties.

C. Back End Development and Results

The backend is a Node.js Express application organized into route modules, each handling a specific domain resource. The server startup sequence validates environment variables (JWT_SECRET minimum 16 characters, DATABASE_URL), runs database migrations from schema.sql, and ensures the admin account exists before listening on the configured port.

The following table summarizes the implemented API endpoints:

Method & Path	**Access**	**Description**	**Key Security**
POST /api/auth/register	Public	Create TENANT account	Rate limit 30/15min, Zod, bcrypt
POST /api/auth/login	Public	Authenticate, set cookie	Rate limit, lockout, timing-safe
POST /api/auth/logout	Auth	Clear session cookie	CSRF header required
GET /api/dashboard/admin	ADMIN	Occupancy, financials, trends	Role middleware
GET /api/dashboard/tenant	TENANT	Personal status, dues	Ownership check
GET/POST/PUT/DELETE /api/rooms	Read: all; Write: ADMIN	Room CRUD	Capacity validation, FK check
GET/PATCH /api/tenants	ADMIN	List, edit, assign rooms	Role, capacity enforcement
GET/POST/PATCH /api/payments	Tenant submit; Admin review	Rent workflow	Magic-byte, ownership, dedup
GET/POST/PATCH /api/complaints	Tenant raise; Admin update	Issue tracking	Isolation by user_id
GET/POST/DELETE /api/notices	Read: all; Write: ADMIN	Announcements	Role middleware
GET/POST/DELETE /api/expenses	ADMIN	Cost tracking	Date format validation

The centralized error handler catches ZodError (returning 400 with the first issue message), file size exceeded (400), invalid image type (400), JSON parse failure (400), and all unhandled errors (500 with a generic message, never exposing stack traces or SQL errors). The audit_log table records every state-changing action with user ID, action label, detail text, IP address, and timestamp, providing a complete forensic trail.

Payment proof images are stored as BYTEA in the payments table and served through a dedicated endpoint that enforces ownership (tenant sees only their own proofs, admin sees all), sets X-Content-Type-Options: nosniff, Cache-Control: private, and a locked Content-Security-Policy header. The CSV export endpoint sanitizes cells against formula injection by prefixing dangerous characters with a quote.

V. RESULT AND ANALYSIS

A. Functional Test Case Analysis

The automated test suite executes 15 integration tests against the full Express application using an in-memory PostgreSQL emulator (pg-mem). Each test creates isolated HTTP clients with independent cookie jars, simulating real browser sessions. All 15 tests pass successfully with a total execution time of 4631 ms.

Test #	Objective	Input / Condition	Expected	Result
1	CSRF and auth enforcement	GET /api/rooms without cookie; POST without X-Requested-With	401; 403; CSP header present	PASS
2	Admin login sets secure cookie	Valid admin credentials	200, httpOnly SameSite=Strict cookie	PASS
3	Password policy and registration	Weak password 'abc'; valid tenant registration	400; 201 with TENANT role	PASS
4	Account lockout	5 wrong passwords then correct password	401 x5, then 423 Locked	PASS
5	RBAC enforcement	Tenant accessing admin endpoints	403 Forbidden	PASS
6	Room capacity	Assign 2nd tenant to capacity-1 room	409 Conflict	PASS
7	File upload validation	Script content with image MIME; valid PNG; duplicate month; future month	400; 201; 409; 400	PASS

B. Security Test Case Analysis

Security-specific tests verify that the layered defences function correctly:

- Payment proof privacy: An unregistered outsider receives 404 when requesting another tenant's proof image; the owning tenant and admin receive 200 with correct content-type.
- Approval workflow integrity: Tenant cannot self-approve (403); admin can approve once (200); re-review of an already-approved payment returns 409.
- Receipt access control: Receipt endpoint returns 409 for non-approved payments; returns formatted receipt data only after approval.
- Immediate deactivation effect: After admin deactivates a tenant, the next authenticated request returns 401 despite the JWT still being within its 8-hour validity window.
- Formula injection in CSV: Cells beginning with =, +, -, or @ are prefixed with a quote character, preventing execution in spreadsheet applications.
- Timing-safe authentication: Unknown email addresses receive a bcrypt comparison against a dummy hash, ensuring response time does not leak whether an email exists.

C. Edge-Case and Failure Analysis

Additional edge cases verified by the test suite and code review:

- Future month payment: Attempting to submit rent for 2999-01 returns 400 with clear error message.
- Duplicate payment: Submitting a second payment for the same month while one is PENDING or APPROVED returns 409, preventing double-billing.

- Room deletion with occupants: Attempting to delete an occupied room returns 409 with guidance to move tenants first.
- Capacity reduction below occupancy: Editing a room's capacity below current tenant count returns 409.
- Deactivated tenant room vacating: Deactivation automatically nullifies the room_id, freeing the bed.
- Invalid date format for expenses: Date string '10/10/2026' (wrong format) returns 400; only YYYY-MM-DD is accepted.

D. Total Performance Analysis

The test suite execution provides measurable performance data. All 15 tests complete in 4631 ms against the in-memory database, with individual test durations ranging from 11.9 ms (RBAC check, primarily middleware evaluation) to 1269.7 ms (lockout test, dominated by 5 bcrypt comparisons at cost factor 12). The bcrypt cost factor intentionally trades CPU time for security; in production with a persistent database, login latency is dominated by the hashing operation rather than I/O.

Key performance characteristics observed from the implementation:

- Authentication latency: bcrypt hash comparison at cost 12 requires approximately 200-300 ms on modern hardware, providing brute-force resistance.
- Rate limiting: 600 requests per 15 minutes per IP prevents abuse while allowing normal multi-tab usage.
- Database pool: Maximum 10 connections with parameterized queries; the application is designed for single-property scale (tens of concurrent users).
- File upload: 2 MB maximum size with memory storage (no disk I/O for temporary files); magic-byte check adds negligible latency (<1 ms for header inspection).
- Response payload limit: 20 KB JSON body cap prevents memory exhaustion from oversized requests.

The system does not implement machine learning inference, so traditional ML metrics (accuracy, precision, recall) are not applicable. Performance is evaluated through functional correctness (15/15 tests passing), security property verification, and response latency within acceptable bounds for an administrative web application.

E. Comparative Analysis

PG Manager is compared against the approaches reported in the surveyed literature on the basis of implemented features and security properties:

Property	**PG Manager**	**Francis [4]**	**Saputra [6]**	**Typical Spreadsheet**
Role separation	Yes (ADMIN/TENANT)	Single role	Single role	N/A
File upload security	Magic-byte + MIME	Not discussed	Not discussed	N/A
CSRF protection	SameSite + custom header	Not discussed	Not discussed	N/A
Audit logging	All actions logged with IP	Not reported	Not reported	None
Cloud deployment	Docker + AWS + HTTPS	Not discussed	Not discussed	N/A

The comparison highlights that PG Manager integrates security hardening and operational features that academic prototypes typically omit. Unlike spreadsheet-based management, the system enforces access control, validates inputs, logs actions, and provides structured dashboards. However, controlled benchmarking against the referenced systems is not feasible because they report no comparable quantitative metrics and use different evaluation contexts.

F. Limitations of Current Work

The following limitations are acknowledged based on the actual implementation:

- No automated UPI payment verification: Payment screenshots are reviewed manually by the admin; the system does not integrate with payment gateway APIs to confirm transactions programmatically.
- No email or SMS notifications: Dues reminders and status updates are visible only within the application; no push notifications or automated communication channels are implemented.
- Single-property scope: The architecture supports one PG property; multi-tenant SaaS operation would require schema changes for property isolation.
- No end-to-end encryption for stored images: Payment proofs are stored as plaintext bytes in PostgreSQL BYTEA columns; database-level encryption at rest depends on the hosting provider.
- Limited test coverage scope: The 15 tests cover security, RBAC, and business logic paths but do not include load testing, frontend unit tests, or accessibility audits.

- No offline capability: The single-page application requires network connectivity; no service worker caching or offline mode is implemented.
- Fixed due-day: The rent due day (5th of each month) is a hardcoded constant requiring code modification to change per-property.

VI. CONCLUSION

This project presented PG Manager, a secure full-stack web application that digitizes the complete administrative workflow for Paying Guest house management. The system addresses the practical problem of managing rooms, tenants, rent collection with proof verification, complaints, notices, and expenses through a unified, access-controlled interface. The implementation employs a React 18 frontend, Node.js/Express backend, and PostgreSQL database, secured through JWT authentication with httpOnly cookies, bcrypt password hashing at cost factor 12, per-account brute-force lockout, CSRF protection via custom headers, Content Security Policy headers, role-based access control with live database verification, Zod input schema validation, parameterized SQL queries, and magic-byte file type verification for uploaded payment screenshots. Data-driven dashboards compute occupancy rates, collection progress, overdue tenant identification, and six-month financial trends for informed administrative decisions. Fifteen automated integration tests validate authentication, authorization, business logic correctness, file upload security, and data isolation between tenants, all passing successfully. The system is deployable through Docker Compose or on AWS infrastructure with HTTPS termination via Nginx and Let's Encrypt. The primary current limitation is the absence of automated payment gateway verification, requiring manual screenshot review by the administrator. Future work could integrate UPI payment confirmation APIs, add email or SMS notification channels, implement multi-property SaaS support, and extend test coverage with performance and accessibility testing.

REFERENCES

[1] A. Bucko, K. Vishi, B. Krasniqi, and B. Rexha, "Enhancing JWT authentication and authorization in web applications based on user behavior history," Computers, vol. 12, no. 4, art. 78, Apr. 2023. DOI: 10.3390/computers12040078.

[2] P. Wichmann, M. Steffens, and E. Bodden, "FileUploadChecker: Detecting and sanitizing malicious file uploads in web applications at the request level," in Proc. ACM Symp. Applied Computing (SAC), 2022, pp. 223–230. DOI: 10.1145/3538969.3538999.

[3] S. Deng, L. Peng, M. Galinina, A. Ilyas, and Q. Zhu, "Cloud-native computing: A survey from the perspective of services," ACM Comput. Surv., vol. 55, no. 6, art. 125, 2023. DOI: 10.1145/3524105.

[4] A. Francis and C. N. Frimpong, "Design and implementation of a house rental management system: A comprehensive approach," Int. J. Novel Res. Dev., vol. 8, no. 12, pp. 553–560, Dec. 2023.

[5] M. Rana, S. Kumar, and R. Sharma, "Enhancing data security: A comprehensive study on the efficacy of JSON Web Token (JWT) and HMAC SHA-256 algorithm for web application security," Int. J. Res. Innov. in Technol. and Comput. Commun., vol. 11, no. 9, Sep. 2023. DOI: 10.17762/ijritcc.v11i9.9930.

[6] R. G. Saputra, A. Nugroho, and D. Setiawan, "Implementation of web-based boarding room management information system," Bulletin of Research and Development, 2025.

[7] S. Khalfaoui, A. Benslimane, and M. Kaddoum, "Microservices-driven automation in full-stack development," IEEE Access, vol. 13, 2025.